# SUPPORT2 Dataset: Data Cleaning & Preparation

**AAI-500 Applied Statistics for AI | Final Team Project**

---

## Notebook Objectives

This notebook covers the **Data Cleaning & Preparation** phase of the project pipeline:

1. Initial data inspection and profiling
2. Column renames per PEP-8
3. Engineering the binary target variable (`death_180d`)
4. Data type casting and ordinal/nominal encoding
5. Missing data analysis and imputation
6. Dropping unused columns
7. Documenting target / feature / outcome
8. Export of the cleaned dataset for EDA and modeling

---

## Section 0: Setup & Imports

In [ ]:
```
import sys
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd


sys.path.append("../../utils")
from dataset import get_project_root, load_csv

warnings.filterwarnings("ignore")

current = Path.cwd().resolve()

while current != current.parent:
    if (current / ".git").exists() or (current / "pyproject.toml").exists():
        project_root = current
        break
    current = current.parent
```

```
PROJ_ROOT = get_project_root()
OUTPUT_PATH = PROJ_ROOT / "data" / "support2_cleaned.csv"
print(f"Output path  : {OUTPUT_PATH}")
```

---

# Section 1: Load Data & Initial Inspection

In [ ]:
```
df_raw = load_csv("support2_raw_complete.csv")
print(f"Shape: {df_raw.shape[0]:,} rows  x  {df_raw.shape[1]} columns")
df_raw.head()
```

In [ ]:
```
df_raw.info(verbose=True, show_counts=True)
```

In [ ]:
```
int_cols = df_raw.select_dtypes(include="int64").columns
binary_cols = [
    c
    for c in int_cols
    if df_raw[c].nunique() == 2 and set(df_raw[c].unique()) == {0, 1}
]
print("===========Binary Columns==========")
print(binary_cols)
```

## 1.1 Descriptive Statistics

In [ ]:
```
desc = df_raw.describe().T
desc.columns = [
    "Count",
    "Mean",
    "Std Dev",
    "Min",
    "Q1 (25%)",
    "Median (50%)",
    "Q3 (75%)",
    "Max",
]
desc
```

---

# Section 2: Column Renaming

Rename some columsn to follow PEP-8 standard

In [ ]:
```
rename_map = {
    "d.time": "d_time",
    "num.co": "num_co",
}
df = df_raw.rename(columns=rename_map).copy()

print("Column renames applied:")
```

```
for old, new in rename_map.items():
    print(f"  {old!r}  ->  {new!r}")
print(f"\nTotal columns: {len(df.columns)}")
```

---

# Section 3: Target Variable

**Did the patient die within 180 days of study?**

To answer this we look at two fields:

- `death` : whether the patient ever died during 180 days (1 = yes, 0 = no)
- `d_time` : days of follow-up (how long they were observed). If this number is greater than 180 days of the trial, that means they survived the trial

death_180d (1 = yes, 0 = no) = {death=1 AND d_time <= 180}

Notice the AND conditional because the patient must've died within the 180 days to satisfy our target. We then use **death_180d** for binary classification

In [ ]:
```
df["death_180d"] = ((df["death"] == 1) & (df["d_time"] <= 180)).astype(int)

counts = df["death_180d"].value_counts()
rel_freqs = df["death_180d"].value_counts(normalize=True)

summary = pd.DataFrame(
    {
        "Outcome": ["Survived >= 180 days (0)", "Died within 180 days (1)"],
        "Frequency": counts.values,
        "Relative Frequency": rel_freqs,
        "Percentage (%)": (rel_freqs.values * 100),
    }
)
print(summary.to_string(index=False))
print(f"\nClass imbalance ratio (survived: died) = {counts[0] / counts[1]:.2
```

---

# Section 4: Data Type Casting (into Pandas Type)

Correct data types are essential for proper computing and modeling. We apply three categories of correction. Casting to Int8 also handles NaN gracefully:

1. **Binary integer columns** -> `Int8` (nullable integer; memory-efficient)
2. **Nominal categorical columns** -> unordered `pd.Categorical`
3. **Ordinal categorical columns** -> ordered `pd.Categorical` for proper sorting and comparision

**`income` ordering (ascending socioeconomic status):** `under $11k` < `$11-$25k` < `$25-$50k` < `>$50k`

**`sfdm2` ordering (best functional status → worst):** `no disability` < `moderate ADL impairment` < `severe SIP impairment` < `coma/intubated` < `died before 2-month interview`

In [ ]:
```
# Binary columns -> nullable Int8
binary_cols = ["death", "hospdead", "diabetes", "dementia"]
for col in binary_cols:
    df[col] = df[col].astype("Int8")

######  8 Catagoricals
# sex = 2 levels
# dzgroup = 5
# dzclass = 4
# race = 5
# income = 4
# ca = 3
# dnr = 3
# sfdm2 = 5
##############

# Nominal (unordered) categoricals
nominal_cats = ["sex", "dzgroup", "dzclass", "race", "ca", "dnr"]
for col in nominal_cats:
    df[col] = df[col].astype("category")

# Ordinal: income (lowest -> highest bracket) as coded in support2_data_dict
income_order = ["under $11k", "$11-$25k", "$25-$50k", ">$50k"]
df["income"] = pd.Categorical(df["income"], categories=income_order, orderec

# Ordinal: sfdm2 (2-month functional disability; no disability -> died befor
# as coded in support2_data_dictionary.md
sfdm2_order = [
    "no(M2 and SIP pres)",  # Level 1 - no moderate/severe disability
    "adl>=4 (>=5 if sur)",  # Level 2 - unable to do 4+ ADL
    "SIP>=30",  # Level 3 - Sickness Impact Profile >= 30
    "Coma or Intub",  # Level 4 - intubated or in coma at 2 months
    "<2 mo. follow-up",  # Level 5 - died before 2-month interview
]
df["sfdm2"] = pd.Categorical(df["sfdm2"], categories=sfdm2_order, ordered=Tr

print("Data type corrections applied:\n")
cols_updated = binary_cols + nominal_cats + ["income", "sfdm2"]
print(df[cols_updated].dtypes.to_string())
```

---

# Section 5: Missing Data Analysis

The authors have stated there are a maximum of 5641 N/As. This, along with every other missing data, we must verify and address.

In [ ]:
```
missing_count = df.isnull().sum()
missing_percent = missing_count / len(df) * 100
missing_df = pd.DataFrame(
    {
        "Missing Count": missing_count,
        "Missing %": missing_percent,
    }
).sort_values("Missing Count", ascending=False)

missing_df = missing_df[missing_df["Missing Count"] > 0]

total_cells = df.shape[0] * df.shape[1]
total_missing = int(missing_df["Missing Count"].sum())

print(f"Columns with at least one missing value : {len(missing_df)}")
print(
    f"Total missing cells : {total_missing:,} / {total_cells:,}  ({total_mis
)
print(missing_df.to_string())

# verify maximum missing n/as
max_na = df_raw.isnull().sum().max()
max_na_col = df_raw.isnull().sum().idxmax()
print(f"Max NAs: {max_na} (column: '{max_na_col}')")
assert max_na == 5641, f"Expected 5641, got {max_na}"
```

In [ ]:
```
fig, ax = plt.subplots(figsize=(11, 6))

bars = ax.barh(
    missing_df.index[::-1],
    missing_df["Missing %"][::-1],
    edgecolor="white",
    height=0.7,
)


ax.set_xlabel("Missing Data (%)", fontsize=11)
ax.set_title("Missing Data Rate by Column", fontsize=13, fontweight="bold")
ax.set_xlim(0, 80)

for bar, pct in zip(bars, missing_df["Missing %"][::-1]):
    if pct > 0:
        ax.text(
            bar.get_width() + 0.5,
            bar.get_y() + bar.get_height() / 2,
            f"{pct:.1f}%",
            va="center",
            fontsize=8,
        )
plt.show()
```

## 5.1 Missing Data Decision Table

Here we decide how to tackle missing data. Either drop the column entirely or impute mathematically

For most of our biological features that have missing data, we compute the median. The reason is per Agresti et al (2022), chapter 3, that for right-skewed data, using median instead of mean worse the skewness of the data (push it woard the tail). Looking at section 1, our columns the mean > median, which signals right skewed.

**Column(s) Decision Rationale**

| `adlp`, `adls` | **Drop** | Superseded by `adlsc` per original authors | `dnrday` | **Drop** | Do-no-resus day. Useless for our purpose | `hday` | **Drop** | Day admitted to study. Nothing to do with mortality | `charges, totcst, totmcst` | **Drop** | Relating to hospital cost, which we won't be studying | `alb`, `pafi`, `bili`, `crea`, `bun`, `wblc`, `urine` | **imputed values avail** | Per Author's note we have the recommended imputed values | `ph`, `glucose`, `scoma`, `avtisst`, `edu`, `prg2m`, `prg6m` | **Sample median** | high missing | | `surv2m`, `surv6m`, `aps`, | **Sample median** | kept as benchmark columns | | `meanbp`, `hrt`, `resp`, `temp`, `sod` | **Sample median** | few missing | | `charges`, `totcst` | **Sample median** | Retained for future cost analysis | | `totmcst` | **Sample median** | Retained for future cost analysis; many missing values and will be difficult to impute. Contains negative values (balances) | | `race`, `income`, `sfdm2`, `dnr` | **Mode imputation** | Most frequent category as best single-value estimate for categoricals |

### Benchmark Columns (kept but will be excluded from death_180d model features)

| Column | Role | Notes |
|---|---|---|
| `surv2m`, `surv6m` | SUPPORT model predictions | will be compared against our model |
| `aps` | APACHE III severity score | Independent benchmark system widely used in critical care |
| `sps` | SUPPORT physiology sub-score | Component of the SUPPORT model ?? |
| `prg2m`, `prg6m` | Physician survival estimates | ML vs. clinician judgment comparison |
| `dnr` | DNR order status | **Should not use as predictor feature as it's reflects human behavior rather than biological factors** |

---

# Section 6: Feature Pruning

We remove features not used in our analysis:

In [ ]:
```
cols_to_drop = ["adlp", "adls", "dnrday", "hday", "charges", "totcst", "totm

df.drop(columns=cols_to_drop, inplace=True, errors="ignore")
print(f"Dropped  : {len(cols_to_drop)} columns")
print(f"Remaining: {df.shape[0]:,} rows  x  {df.shape[1]} columns")
print(f"Remaining columns ({df.shape[1]}):")
print(list(df.columns))
```

---

# Section 7: Missing Data Imputation

The original SUPPORT study authors (Harrell, 2022) provide clinically validated **normal fill-in values** for seven physiological variables (see docs/support2_dataset_description.md). We will use this for imputation

| Variable | Normal Fill | Meaning |
|---|---|---|
| `alb` | 3.5 | Serum albumin - liver / nutrition |
| `pafi` | 333.3 | PaO2/FiO2 ratio - oxygen level |
| `bili` | 1.01 | Bilirubin - liver function |
| `crea` | 1.01 | Creatinine - renal function |
| `bun` | 6.51 | BUN - normal renal function |
| `wblc` | 9.0 | WBC count - immune status |
| `urine` | 2502 | Urine output - renal output |

In [ ]:
```
DOMAIN_FILL = {
    "alb": 3.5,
    "pafi": 333.3,
    "bili": 1.01,
    "crea": 1.01,
    "bun": 6.51,
    "wblc": 9.0,
    "urine": 2502.0,
}

imputation_log = []

for k, v in DOMAIN_FILL.items():
    n_missing = int(df[k].isnull().sum())
```

```
    if n_missing > 0:
        df[k] = df[k].fillna(v)
        imputation_log.append(
            {
                "Column": k,
                "Strategy": "Domain Fill",
                "Fill Value": v,
                "Cells Filled": n_missing,
            }
        )

MEDIAN_COLS = [
    "meanbp",
    "hrt",
    "resp",
    "temp",
    "sod",
    "ph",
    "glucose",
    "scoma",
    "avtisst",
    "edu",
    "surv2m",
    "surv6m",
    "aps",
    "prg2m",
    "prg6m",
]
for k in MEDIAN_COLS:
    n_missing = int(df[k].isnull().sum())
    if n_missing > 0:
        med = float(df[k].median())
        df[k] = df[k].fillna(med)
        imputation_log.append(
            {
                "Column": k,
                "Strategy": "Sample Median",
                "Fill Value": round(med, 4),
                "Cells Filled": n_missing,
            }
        )

MODE_COLS = ["race", "income", "sfdm2", "dnr"]

for k in MODE_COLS:
    n_missing = int(df[k].isnull().sum())
    if n_missing > 0:
        mode_val = df[k].mode()[0]
        df[k] = df[k].fillna(mode_val)
        imputation_log.append(
            {
                "Column": k,
                "Strategy": "Sample Mode",
                "Fill Value": str(mode_val),
                "Cells Filled": n_missing,
            }
```

```
        )

log_df = pd.DataFrame(imputation_log)
print("Imputation Summary:")
print(log_df.to_string(index=False))
print(f"Total cells imputed: {log_df.get('Cells Filled', np.array([])).sum()
```

In [ ]:
```
remaining_na = df.isnull().sum()
remaining_na = remaining_na[remaining_na > 0]

if len(remaining_na) == 0:
    print("Imputation complete, zero missing values remain in any column.")
else:
    print("WARNING: Remaining missing values:")
    print(remaining_na.to_string())

print(
    f"\nDataset shape after imputation: {df.shape[0]:,} rows  x  {df.shape[1
)
```

---

# Section 8: Defining Feature, Outcome Separation, Leakage

| Category | Columns |
|---|---|
| **Target** | `death_180d` |
| **Outcome columns** (excluded due to result leakage) | `death` , `hospdead` , `d_time` , `slos` |
| **Row identifier** | `id` |
| **Features** | All remaining columns |

---

# Section 9: Post-Cleaning Summary

We compare the dataset before and after cleaning and present the five-number summary for key physiological lab variables to confirm that imputation and outlier retention produced a coherent, analysis-ready dataset.

In [ ]:
```
print("=== Pre-Cleaning ===")
print(f"  Shape          : {df_raw.shape[0]:,} rows  x  {df_raw.shape[1]} co
print(f"  Missing cells  : {df_raw.isnull().sum().sum():,}")

print("\n=== Post-Cleaning ===")
print(f"  Shape          : {df.shape[0]:,} rows  x  {df.shape[1]} columns")
print(f"  Missing cells  : {df.isnull().sum().sum()}")
print(f"  Rows retained  : {df.shape[0] / df_raw.shape[0] * 100:.1f}%")
```

```
prev = df["death_180d"].mean()
```

### 9.1 Important Lab Variables Summary (Post-Cleaning)

In [ ]:
```
key_labs = [
    "age",
    "meanbp",
    "hrt",
    "temp",
    "alb",
    "bili",
    "crea",
    "glucose",
    "bun",
    "urine",
]
post_desc = df[key_labs].describe().T
post_desc.columns = ["Count", "Mean", "Std Dev", "Min", "Q1", "Median", "Q3"
post_desc[["Min", "Q1", "Median", "Q3", "Max", "Mean", "Std Dev"]]
```

---

## Section 10: Save Cleaned Dataset

In [ ]:
```
df.to_csv(OUTPUT_PATH, index=False)
print(f"Cleaned dataset saved to : {OUTPUT_PATH}")
print(f"Final shape              : {df.shape[0]:,} rows  x  {df.shape[1]} co
print("\nColumn list and dtypes:")
print(df.dtypes.to_string())
```

# SUPPORT2 Dataset: Exploratory Data Analysis

**AAI-500 Applied Statistics for AI | Final Team Project**

---

## Notebook Objectives

This notebook covers the **Exploratory Data Analysis** phase of the project pipeline:

1. Detect Outliers
2. Univariate Analysis
3. Bivariate Analsysis
4. Demographic Analsysis
5. Disease Group Analysis

6. DNR Status Analysis

---

# Learnings from the 1990s study

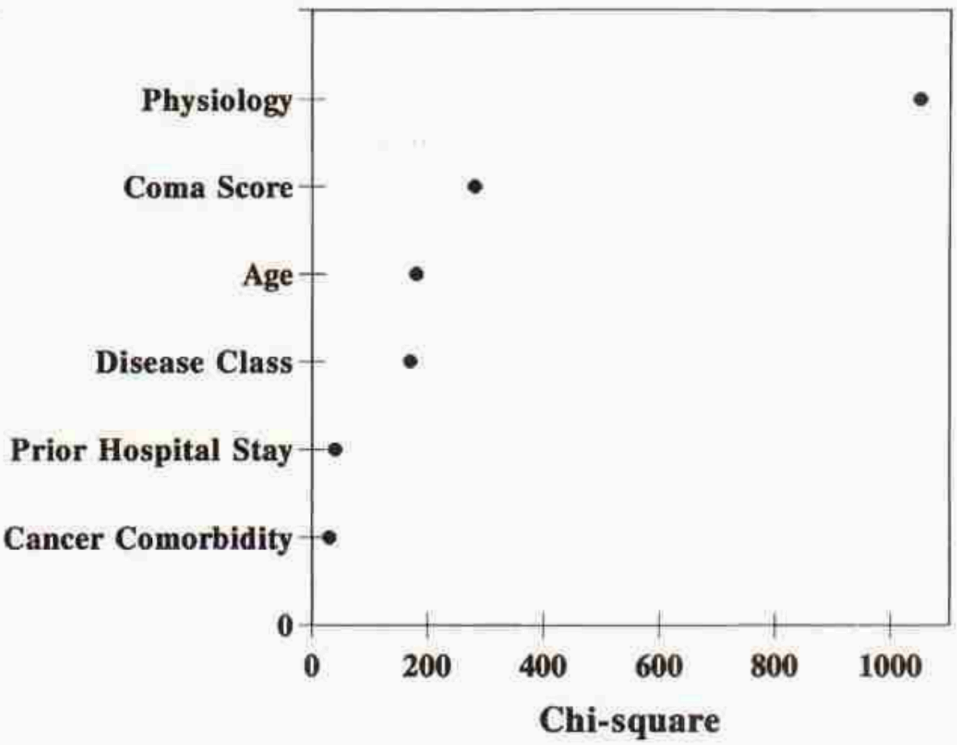


**Figure 1. Relative contributions of major prognostic elements in the SUPPORT model as measured by the amount of chi-square accounted for by each element.** Physiology = all physiologic measures except Glasgow coma scale; cancer comorbidity = presence of cancer in addition to disease category; previous hospital stay = days in the hospital before study entry.

To inform our work, we review Knaus et al's (1995) paper. According to Knaus et al these features were most weighted in their model:

- Physiology Score (calculated from wbc, albumin, etc). Some salient ones are Blood Pressure and Oxygenation levels
- Neurological Function (Glasgow coma scale)
- Patient age (but only when coupled with certain diseases like COPD)
- Apache III score
- Disease-Specific Interactions: The mathematical weight of certain features was explicitly programmed to vary based on the patient's primary

disease:Albumin x Disease: A low albumin level strongly predicted death in cancer, COPD, and congestive heart failure, but was relatively unimportant in coma or MOSF. WBC x Disease: A low WBC count was specifically associated with a greater risk of death in acute respiratory failure and MOSF. Age x Disease: Advancing age heavily impacted short-term survival in COPD but had almost no incremental effect on patients with severe acute complications like MOSF or malignancy.

---

# Section 0: Setup & Imports

In [ ]:
```
import sys
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

_here = Path.cwd()
_proj_root = _here if (_here / "utils").exists() else (_here / "../..").resc
if str(_proj_root) not in sys.path:
    sys.path.insert(0, str(_proj_root))

from utils.dataset import load_csv  # noqa: E402

warnings.filterwarnings("ignore")
```

---

# Section 1: Load Cleaned Data

In [ ]:
```
df = load_csv("support2_cleaned.csv")
print(f"Shape: {df.shape[0]:,} rows  x  {df.shape[1]} columns")

OUTCOME_COLS = ["death", "hospdead", "d_time", "slos"]
ID_COLS = ["id"]
TARGET_COL = "death_180d"
BENCHMARK_COLS = ["surv2m", "surv6m", "aps", "sps", "prg2m", "prg6m", "dnr"]
LAB_COLS = [
    "meanbp",
    "wblc",
    "hrt",
    "resp",
    "temp",
    "pafi",
    "alb",
    "bili",
    "crea",
    "sod",
```

```
    "ph",
    "glucose",
    "bun",
    "urine",
]
FEATURE_COLS = [
    c
    for c in df.columns
    if c not in OUTCOME_COLS + ID_COLS + [TARGET_COL] + BENCHMARK_COLS
]

survivors = df[df[TARGET_COL] == 0]
died = df[df[TARGET_COL] == 1]
overall_rate = df[TARGET_COL].mean()
print(f"Survived 180d   : {len(survivors):,}  ({1 - overall_rate:.1%})")
print(f"Died within 180d: {len(died):,}  ({overall_rate:.1%})")
```

---

# Section 2: Physiology Outlier Detection (IQR Method)

```
IQR = Q3 - Q1
Lo whisker = Q1 - 1.5IQR
Hi whisker = Q3 + 1.5IQR
```

- Urine boxplot median is pushed all the way right because of imputation with Norm-Fill.
- Glucose boxplot collapses due to high number of imputed NAs.
- **Probably better to run outliers before imputation**

In [ ]:
```
def compute_iqr_outliers(df, cols):
    rows = []
    for col in cols:
        Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
        IQR = Q3 - Q1
        lo, hi = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
        outliers = int(((df[col] < lo) | (df[col] > hi)).sum())
        rows.append(
            {
                "Column": col,
                "Q1": round(float(Q1), 3),
                "Q3": round(float(Q3), 3),
                "IQR": round(float(IQR), 3),
                "Lo Whisker": round(float(lo), 3),
                "Hi Whisker": round(float(hi), 3),
                "Outliers": outliers,
                "Outlier %": round(outliers / len(df) * 100, 1),
            }
        )
    return pd.DataFrame(rows).sort_values("Outlier %", ascending=False)
```

```
outlier_df = compute_iqr_outliers(df, LAB_COLS)
print(
    outlier_df[
        ["Column", "Lo Whisker", "Hi Whisker", "Outliers", "Outlier %"]
    ].to_string(index=False)
)
```

In [ ]:
```
top6 = outlier_df.head(6)["Column"].tolist()
fig, axes = plt.subplots(2, 3, figsize=(14, 7))
for i, col in enumerate(top6):
    axes.flat[i].boxplot(
        df[col],
        vert=False,
        medianprops=dict(color="red", linewidth=2),
        flierprops=dict(marker="o", alpha=0.2, markersize=3, color="gray"),
    )
    axes.flat[i].set_title(col)
    axes.flat[i].set_xlabel("Value")
fig.suptitle("Top 6 High Outliers Lab Variables", fontsize=12, fontweight="b
plt.tight_layout()
plt.show()
```

---

# Section 3: Univariate Distributions

- Lab features are mainly right skewed
- Some imputations cause spikeness (glucose, pafi, bili)

In [ ]:
```
fig, axes = plt.subplots(4, 4, figsize=(30, 25))
for i, col in enumerate(LAB_COLS):
    axes.flat[i].hist(df[col], bins=50, edgecolor="white")
    axes.flat[i].set_title(col)
    axes.flat[i].set_xlabel("Value")
    axes.flat[i].set_ylabel("Count")
for j in range(len(LAB_COLS), len(axes.flat)):
    axes.flat[j].set_visible(False)
fig.suptitle(
    "Univariate Distributions - Day-3 Lab Variables", fontsize=13, fontweigh
)
plt.tight_layout()
plt.show()
```

## 3.1 Age follows normal distribution

In [ ]:
```
fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(df["age"], bins=40, edgecolor="white")

ax.set_title("Age Distribution")
plt.show()
```

---

# Section 4: Bivariate Analysis - Lab Values by Survival Outcome

- Difficult to see any impact these lab values make with this analysis
- The paper did say this only becomes evident when combined with Disease group. For example, a high creatine with kidney failure gives high likelihood of mortality.

In [ ]:
```
fig, axes = plt.subplots(4, 4, figsize=(16, 14))
for i, col in enumerate(LAB_COLS):
    ax = axes.flat[i]
    sns.boxplot(
        data=df,
        y=TARGET_COL,
        x=col,
        ax=ax,
        orient="h",
        palette=["lightgreen", "pink"],
        width=0.5,
        flierprops=dict(marker=".", alpha=0.2, markersize=3),
    )
    ax.set_title(col, fontsize=10)
    ax.set_xlabel("Value")
    ax.set_ylabel("")
    ax.set_yticks([0, 1])
    ax.set_yticklabels(["Survived", "Died"])

for j in range(len(LAB_COLS), len(axes.flat)):
    axes.flat[j].set_visible(False)

fig.suptitle(
    "Lab Values by Survival Outcome",
    fontsize=12,
    fontweight="bold",
)
plt.tight_layout()
plt.show()
```

---

# Section 5: Demographic Analysis

- Mortality gradual decline as wealth increases
- Asian group a bit more mortality

In [ ]:
```
def plot_mortality_rate(df, col, target_col, overall_rate, ax, title=None, s
    rate = df.groupby(col, observed=True)[target_col].mean()
    rate = rate.sort_values(ascending=False)
```

```
    bars = ax.bar(
        range(len(rate)),
        rate.values * 100,
        edgecolor="white",
        width=0.6,
    )
    ax.axhline(
        overall_rate * 100,
        color="black",
        linestyle="--",
        linewidth=1.2,
        label=f"Overall ({overall_rate:.1%})",
    )
    ax.set_title(title or f"Mortality Rate by {col}")
    ax.set_ylabel("180-Day Mortality (%)")
    ax.set_xticks(range(len(rate)))
    ax.set_xticklabels(rate.index, rotation=30, ha="right", fontsize=8)
    ax.legend(fontsize=8)
    for bar, v in zip(bars, rate.values):
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.5,
            f"{v:.1%}",
            ha="center",
            fontsize=8,
        )
```

```
# education bins
edu_bins = [0, 8, 12, 15, 100]
edu_labels = ["0-8 yrs", "9-12 yrs", "13-15 yrs", "16+ yrs"]
df["edu_group"] = pd.cut(df["edu"], bins=edu_bins,
labels=edu_labels, right=True)

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

plot_mortality_rate(
    df, "edu_group", TARGET_COL, overall_rate, axes[0, 0],
title="Education", sort=False
)
plot_mortality_rate(df, "income", TARGET_COL, overall_rate, axes[0,
1], title="Income")
plot_mortality_rate(df, "race", TARGET_COL, overall_rate, axes[0,
2], title="Race")
plot_mortality_rate(df, "sex", TARGET_COL, overall_rate, axes[1,
0], title="Sex")
plot_mortality_rate(
    df, "ca", TARGET_COL, overall_rate, axes[1, 1], title="Cancer
Status"
)
axes[1, 2].set_visible(False)

fig.suptitle(
    "Section 5: 180-Day Mortality Rate by Demographic & Categorical
Features",
```

```
    fontsize=12,
    fontweight="bold",
)
plt.tight_layout()
plt.show()

# Survivorship by age
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.boxplot(data=df, x=TARGET_COL, y="age", ax=axes[0], palette=
["green", "red"])
axes[0].set_title("Age by Survival Outcome")
axes[0].set_xticklabels(["Survived", "Died"])
axes[0].set_xlabel("")
axes[0].set_ylabel("Age (years)")

# KDE overlay
for label, grp, color in [("Survived", survivors, "green"),
("Died", died, "red")]:
    grp["age"].plot.kde(ax=axes[1], label=label, color=color,
linewidth=2)
axes[1].axvline(survivors["age"].mean(), color="green",
linestyle="--", linewidth=1)
axes[1].axvline(died["age"].mean(), color="red", linestyle="--",
linewidth=1)
axes[1].set_title("Age Distribution by Survival Outcome")
axes[1].set_xlabel("Age (years)")
axes[1].set_ylabel("Density")
axes[1].legend()

plt.tight_layout()
plt.show()
```

---

## Section 6: Disease Group Analysis

In [ ]:

```
# mortality rate (uses utility) + patient count annotation
grp = df.groupby("dzgroup", observed=True)["death_180d"].agg(["mean", "count
grp.columns = ["Mortality", "N"]
grp = grp.sort_values("Mortality", ascending=False)

fig, ax = plt.subplots(figsize=(10, 5))
plot_mortality_rate(
    df,
    "dzgroup",
    TARGET_COL,
    overall_rate,
    ax,
    title="180-Day Mortality Rate by Disease Group",
)
# annotate patient counts below percentage labels
```

```
for i, (_, row) in enumerate(grp.iterrows()):
    ax.text(
        i,
        row["Mortality"] * 100 - 3.5,
        f"n={row['N']:,}",
        ha="center",
        fontsize=7,
        color="white",
        fontweight="bold",
    )
plt.tight_layout()
plt.show()
```

---

## Section 7: DNR Status

- **no dnr**: no order placed
- **dnr before sadm**: order existed before study admission
- **dnr after sadm**: order placed during the admission (possibly due to detereorating conditions)

In [ ]:
```
DNR_ORDER = ["no dnr", "dnr before sadm", "dnr after sadm"]

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

counts = df["dnr"].value_counts().reindex(DNR_ORDER)

# DNR count
axes[0].bar(range(3), counts.values, edgecolor="white", width=0.6)
axes[0].set_xticks(range(3))
axes[0].set_xticklabels(DNR_ORDER, rotation=15, ha="right", fontsize=8)
axes[0].set_title("Patient Count by DNR Status")
axes[0].set_ylabel("Count")
for i, v in enumerate(counts.values):
    axes[0].text(i, v + 30, f"{v:,}", ha="center", fontsize=9)

# DNR status compare
mort = df.groupby("dnr", observed=True)["death_180d"].mean().reindex(DNR_ORD
hosp = df.groupby("dnr", observed=True)["hospdead"].mean().reindex(DNR_ORDEF
x = range(3)

axes[1].bar(
    [i - 0.2 for i in x],
    mort.values * 100,
    width=0.35,
    color="steelblue",
    edgecolor="white",
    label="180-day mortality",
)
axes[1].bar(
    [i + 0.2 for i in x],
    hosp.values * 100,
```

```
    width=0.35,
    color="lightblue",
    edgecolor="white",
    label="Died in hospital",
)
axes[1].axhline(
    overall_rate * 100,
    color="black",
    linestyle="--",
    linewidth=1.2,
    label=f"Overall mortality ({overall_rate:.1%})",
)
axes[1].set_xticks(list(x))
axes[1].set_xticklabels(DNR_ORDER, rotation=15, ha="right", fontsize=8)
axes[1].set_title("Mortality & Hospital Death Rate by DNR Status")
axes[1].set_ylabel("Rate (%)")
axes[1].legend(fontsize=8)

plt.tight_layout()
plt.show()
```

In [ ]:
```
# DNR rate by Disease Group
dnr_rate = (
    df[df["dnr"] != "no dnr"].groupby("dzgroup", observed=True).size()
    / df.groupby("dzgroup", observed=True).size()
)
dnr_rate = dnr_rate.sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(10, 4))
bars = ax.bar(
    range(len(dnr_rate)),
    dnr_rate.values * 100,
    edgecolor="white",
    width=0.6,
)
ax.set_xticks(range(len(dnr_rate)))
ax.set_xticklabels(dnr_rate.index, rotation=20, ha="right", fontsize=9)
ax.set_ylabel("% of Group with DNR Order")
ax.set_title("DNR Order Rate by Disease Group", fontweight="bold")
plt.show()
```

---

# Section 8: Categorical Features: Chi-Squared Tests & Contingency Tables

Tests whether a categorical variable is **statistically independent** of 180-day mortality.

$$H_0 : \text{the variable is independent of death\_180d}$$

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

Reject $H_0$ when $p < 0.05$.

**Standardized residuals** = $(O - E)/\sqrt{E}$ — cells with $|r| > 2$ deviate significantly from independence and drive the chi-squared result.

## 8.1 Race and Sex shows low P significance

In [ ]:
```
import numpy as np
from scipy.stats import chi2_contingency

CAT_COLS = ["sex", "dzgroup", "dzclass", "race", "ca", "dnr"]

rows = []
for col in CAT_COLS:
    ct = pd.crosstab(df[col], df[TARGET_COL])
    chi2, p, dof, _ = chi2_contingency(ct)
    rows.append(
        {
            "Variable": col,
            "Chi2 Stat": round(chi2, 2),
            "df": dof,
            "p-value": f"{p:.2e}",
            "Significant": "Yes" if p < 0.05 else "No",
        }
    )

chi2_df = pd.DataFrame(rows).sort_values("Chi2 Stat", ascending=False)
print("Chi-Squared Test Summary:")
print(chi2_df.to_string(index=False))
```

## 8.2 Within Disease Group and Cancer, which class affects mortality most?

- Disease Group: MOSF w/ malignant, Coma, Lung Cancer (in that order) die more than overall rate
- Cancer: metatastic patients die more than the overall rate

In [ ]:
```
# Standardized residuals heatmaps
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

for ax, col in zip(axes, ["dzgroup", "ca"]):
    ct = pd.crosstab(df[col], df[TARGET_COL])
    ct.columns = ["Survived", "Died"]
    _, _, _, exp = chi2_contingency(ct)
    std_resid = (ct.values - exp) / np.sqrt(exp)
    std_resid_df = pd.DataFrame(std_resid, index=ct.index, columns=ct.columr

    im = ax.imshow(std_resid_df.values, cmap="RdBu_r", vmin=-6, vmax=6, aspe
    ax.set_xticks(range(len(ct.columns)))
    ax.set_xticklabels(ct.columns)
    ax.set_yticks(range(len(ct.index)))
```

```
    ax.set_yticklabels(ct.index, fontsize=9)
    ax.set_title(
        f"Standardized Residuals: {col}\n|r| > 2 drives chi-squared",
        fontweight="bold",
    )

    for i in range(std_resid_df.shape[0]):
        for j in range(std_resid_df.shape[1]):
            v = std_resid_df.iloc[i, j]
            ax.text(
                j,
                i,
                f"{v:.1f}",
                ha="center",
                va="center",
                fontsize=9,
                color="black" if abs(v) < 3 else "white",
                fontweight="bold",
            )

    plt.colorbar(im, ax=ax, label="Std. Residual")

fig.suptitle(
    "Standardized Residuals: Cells Driving Chi-Squared",
    fontsize=12,
    fontweight="bold",
)
plt.tight_layout()
plt.show()
```

In [ ]:

```
decisions = pd.DataFrame(
    [
        {
            "Feature": "sex",
            "Decision": "Drop",
            "Signal": "Weak",
            "Rationale": "~1-2% mortality gap between male/female; barely ab
        },
        {
            "Feature": "edu",
            "Decision": "Drop",
            "Signal": "Weak",
            "Rationale": "Weak gradient; 18% of values imputed (median=12 ir
        },
        {
            "Feature": "hday",
            "Decision": "Drop",
            "Signal": "Weak",
            "Rationale": "Day entered study; no clear causal link to 180-day
        },
        {
            "Feature": "charges",
            "Decision": "Drop",
            "Signal": "None",
            "Rationale": "Hospital charges — administrative, not a clinical
        },
```

```
        {
            "Feature": "totcst",
            "Decision": "Drop",
            "Signal": "None",
            "Rationale": "Total RCC cost — administrative, not a clinical pr
        },
        {
            "Feature": "totmcst",
            "Decision": "Drop",
            "Signal": "None",
            "Rationale": "Total micro-cost — administrative, 38% imputed",
        },
        {
            "Feature": "All day-3 labs",
            "Decision": "Keep",
            "Signal": "Strong",
            "Rationale": "Primary clinical predictors per Knaus et al. (1995
        },
        {
            "Feature": "age, adlsc, scoma",
            "Decision": "Keep",
            "Signal": "Strong",
            "Rationale": "Strong EDA effect sizes and clear clinical mechani
        },
        {
            "Feature": "dzgroup / dzclass",
            "Decision": "Keep",
            "Signal": "Strong",
            "Rationale": "Largest mortality spread of any categorical featur
        },
        {
            "Feature": "ca, diabetes, dementia",
            "Decision": "Keep",
            "Signal": "Moderate",
            "Rationale": "Comorbidities with known mortality associations",
        },
        {
            "Feature": "income, race",
            "Decision": "Keep",
            "Signal": "Moderate",
            "Rationale": "Socioeconomic signal; moderate mortality gradient
        },
    ]
)

print("Feature Selection Decisions:")
print(decisions.to_string(index=False))
```

In [ ]:

```
decisions = pd.DataFrame(
    [
        {
            "Feature": "sex",
            "Decision": "Drop",
            "Signal": "Weak",
            "Rationale": "~1-2% mortality gap between male/female; barely ab
        },
```

```
        {
            "Feature": "edu",
            "Decision": "Drop",
            "Signal": "Weak",
            "Rationale": "Weak gradient; 18% of values imputed (median=12 ir
        },
        {
            "Feature": "hday",
            "Decision": "Drop",
            "Signal": "Weak",
            "Rationale": "Day entered study; no clear causal link to 180-day
        },
        {
            "Feature": "charges",
            "Decision": "Drop",
            "Signal": "None",
            "Rationale": "Hospital charges — administrative, not a clinical
        },
        {
            "Feature": "totcst",
            "Decision": "Drop",
            "Signal": "None",
            "Rationale": "Total RCC cost — administrative, not a clinical pr
        },
        {
            "Feature": "totmcst",
            "Decision": "Drop",
            "Signal": "None",
            "Rationale": "Total micro-cost — administrative, 38% imputed",
        },
        {
            "Feature": "All day-3 labs",
            "Decision": "Keep",
            "Signal": "Strong",
            "Rationale": "Primary clinical predictors per Knaus et al. (1995
        },
        {
            "Feature": "age, adlsc, scoma",
            "Decision": "Keep",
            "Signal": "Strong",
            "Rationale": "Strong EDA effect sizes and clear clinical mechani
        },
        {
            "Feature": "dzgroup / dzclass",
            "Decision": "Keep",
            "Signal": "Strong",
            "Rationale": "Largest mortality spread of any categorical featur
        },
        {
            "Feature": "ca, diabetes, dementia",
            "Decision": "Keep",
            "Signal": "Moderate",
            "Rationale": "Comorbidities with known mortality associations",
        },
        {
            "Feature": "income, race",
```

```
            "Decision": "Keep",
            "Signal": "Moderate",
            "Rationale": "Socioeconomic signal; moderate mortality gradient
        },
    ]
)

print("Feature Selection Decisions:")
print(decisions.to_string(index=False))
```

---

# Section 9: Additional Feature Pruning after EDA

Base on our EDA we are opting to drop the following features:

| Column(s) | Decision | Rationale |
|---|---|---|
| sex | **Drop** | Weak signifance base on Bivariate and Chi-Squared analysis |
| edu | **Drop** | Weak signifance base on Bivariate and Chi-Squared analysis |

In [ ]:
```
DROP_COLS = ["sex", "edu"]

MODEL_FEATURE_COLS = [c for c in FEATURE_COLS if c not in DROP_COLS]

print(f"Features available in FEATURE_COLS : {len(FEATURE_COLS)}")
print(f"Features dropped                   : {len(DROP_COLS)}  {DROP_COLS}")
print(f"Final model feature count          : {len(MODEL_FEATURE_COLS)}")
print()
print("Final model features:")
for col in MODEL_FEATURE_COLS:
    print(f"  {col}")
```

In [ ]:
```
OUTPUT_MODEL = _proj_root / "data" / "support2_model_features.csv"
df[MODEL_FEATURE_COLS + [TARGET_COL]].to_csv(OUTPUT_MODEL, index=False)

n_feat = len(MODEL_FEATURE_COLS)
print(f"Saved : {OUTPUT_MODEL}")
print(f"Shape : {df.shape[0]:,} rows  x  {n_feat + 1} columns")
print(f"  Features : {n_feat}  |  Target : {TARGET_COL}")
print()
print("Data layers:")
print("  Bronze : support2_raw_complete.csv  (raw download)")
print("  Silver : support2_cleaned.csv       (imputed, all columns)")
print("  Gold   : support2_model_features.csv (feature-selected, model-ready
```